

Conecta +

Plataforma para gestionar donaciones de alimentos y recursos entre empresas y organizaciones sociales

ALUMNO

Evan David Morales
Serrano

MATRÍCULA

AL03086130

PROFESOR

Santos Guadalupe Facio
Barraza

MATERIA

Ingeniería de
Software

EL PROBLEMA

A unas empresas les sobran alimentos y recursos que otras organizaciones sociales necesitan. Falta un canal confiable que las conecte.



Excedentes que caducan

Productos en buen estado que no llegan a nadie a tiempo.



Sin visibilidad

Las organizaciones no saben qué hay disponible ni dónde.



Sin trazabilidad

Nadie registra quién donó, quién recibió y cuándo.

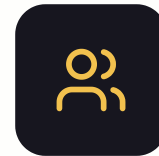
LA SOLUCIÓN

Una red con tres roles



Donador

Publica alimentos o recursos, sigue su estado y ve su impacto: kilos entregados y organizaciones apoyadas.



Beneficiario

Organización social verificada que busca, solicita y confirma la recepción de donaciones.

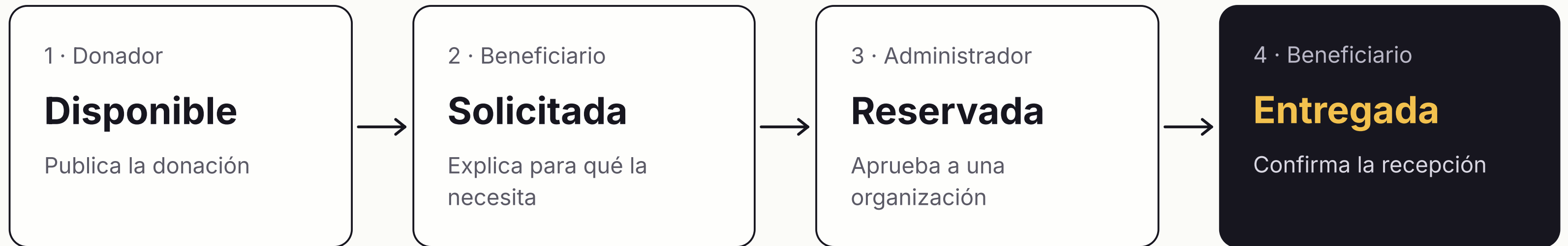


Administrador

Verifica organizaciones, asigna cada donación y supervisa la red con métricas y auditoría.

FLUJO

El ciclo de una donación



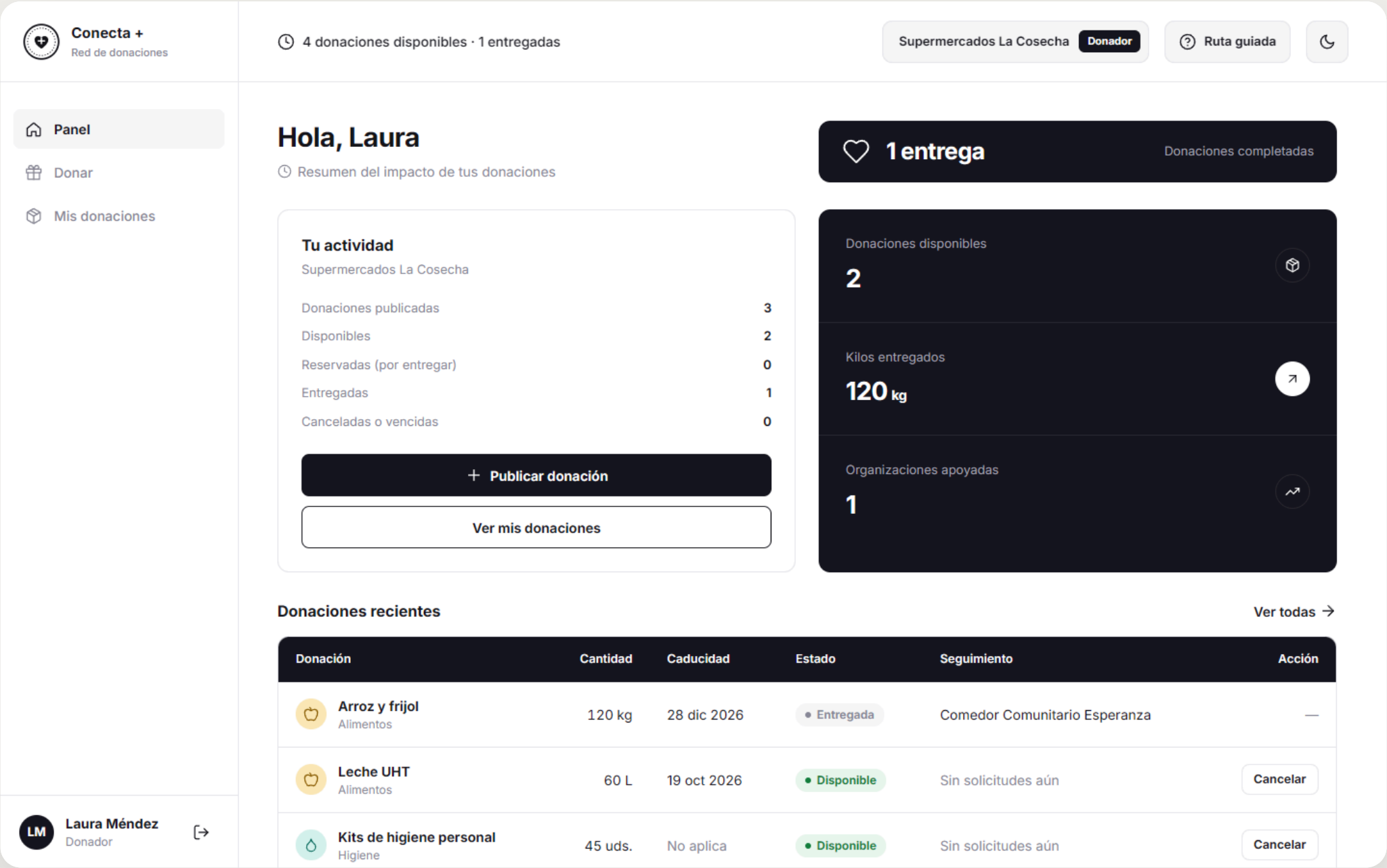
⚡ Al aprobar una solicitud, las demás de esa donación se rechazan solas.

🕒 Si un alimento caduca sin asignarse, pasa a «Vencida» automáticamente.

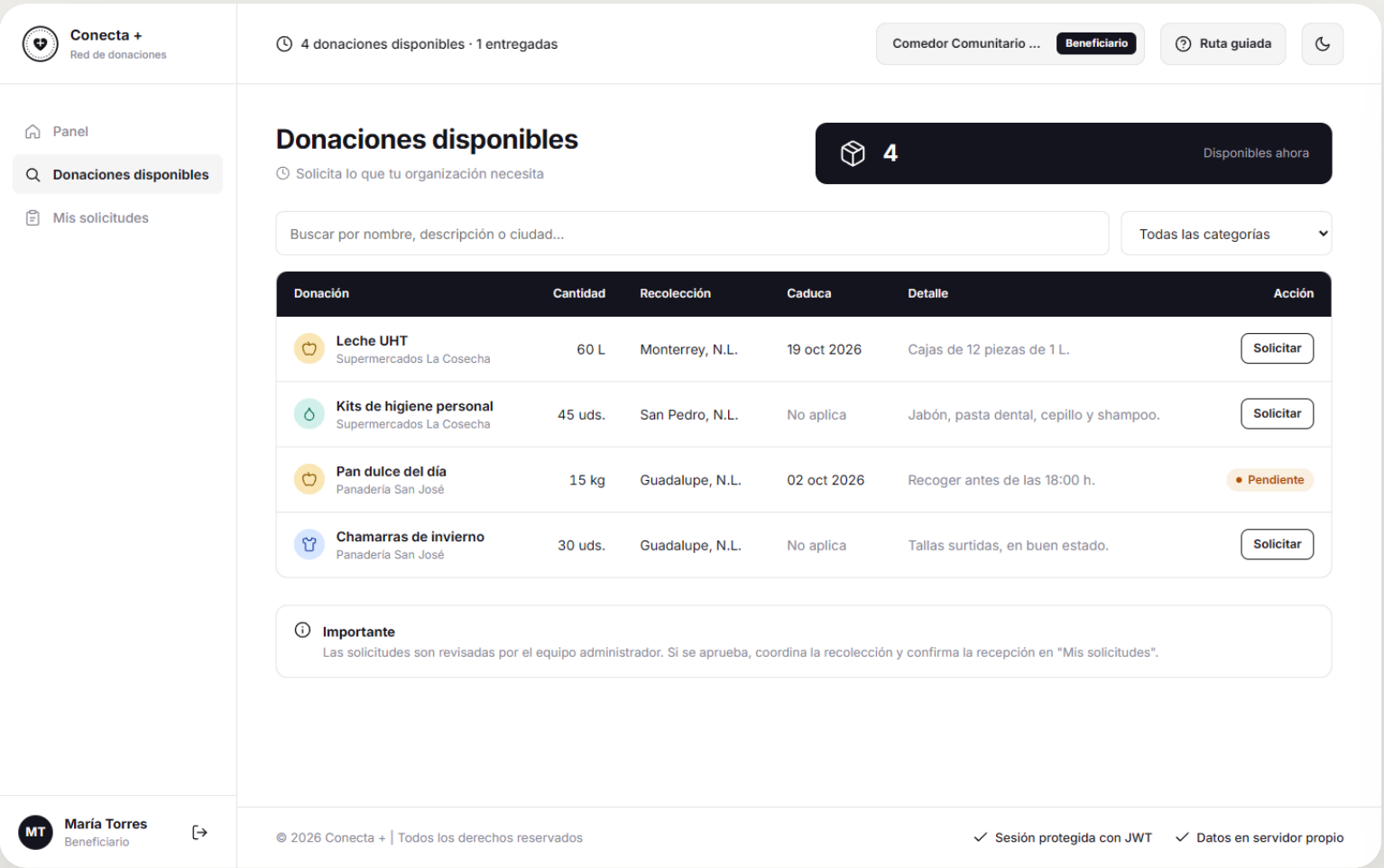
LA APLICACIÓN

Panel del donador

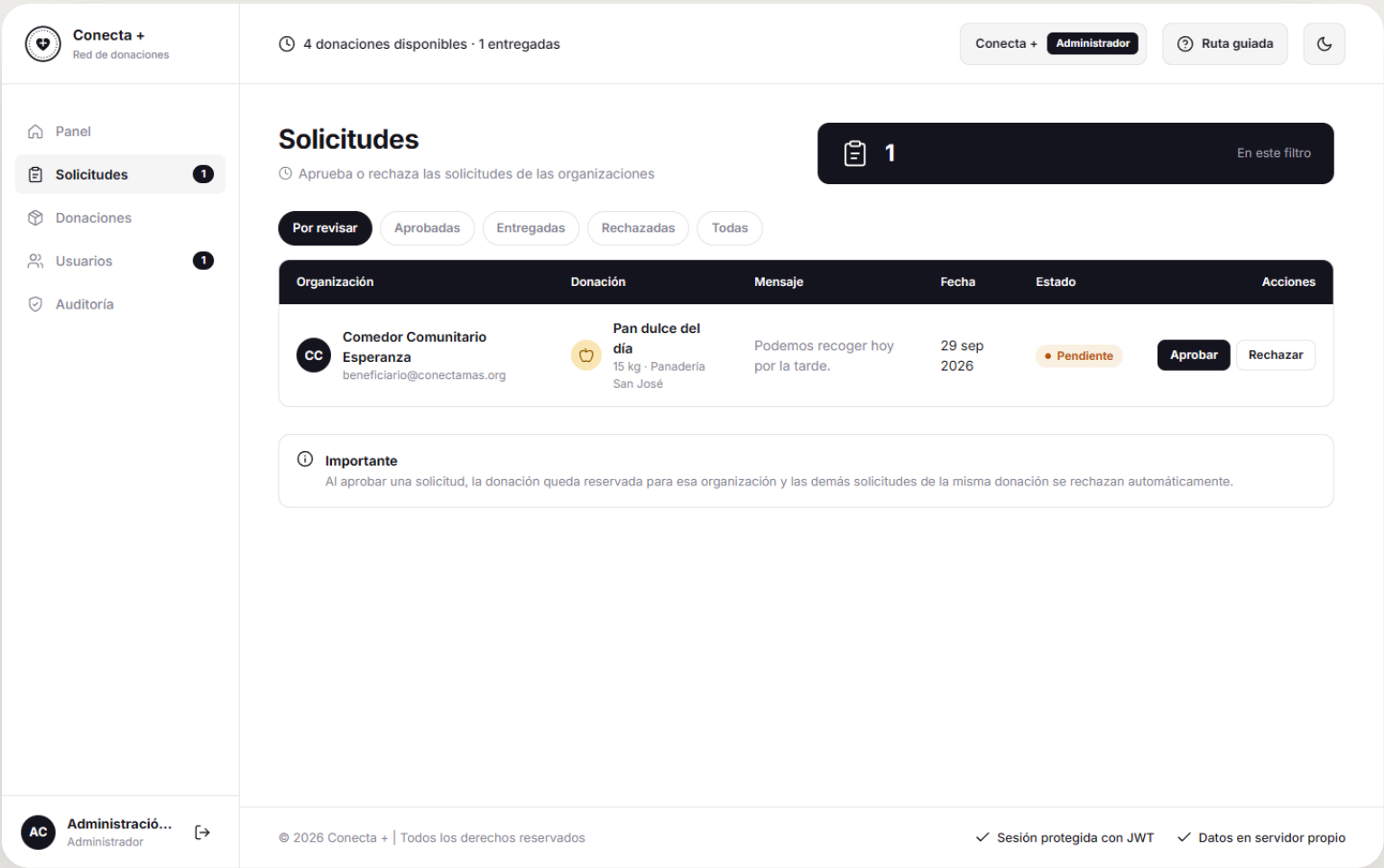
- Impacto en kilos y entregas
- Resumen por estado
- Publicar en un formulario
- Diseño basado en la referencia



Beneficiario y administrador



Beneficiario: busca, filtra y solicita donaciones.



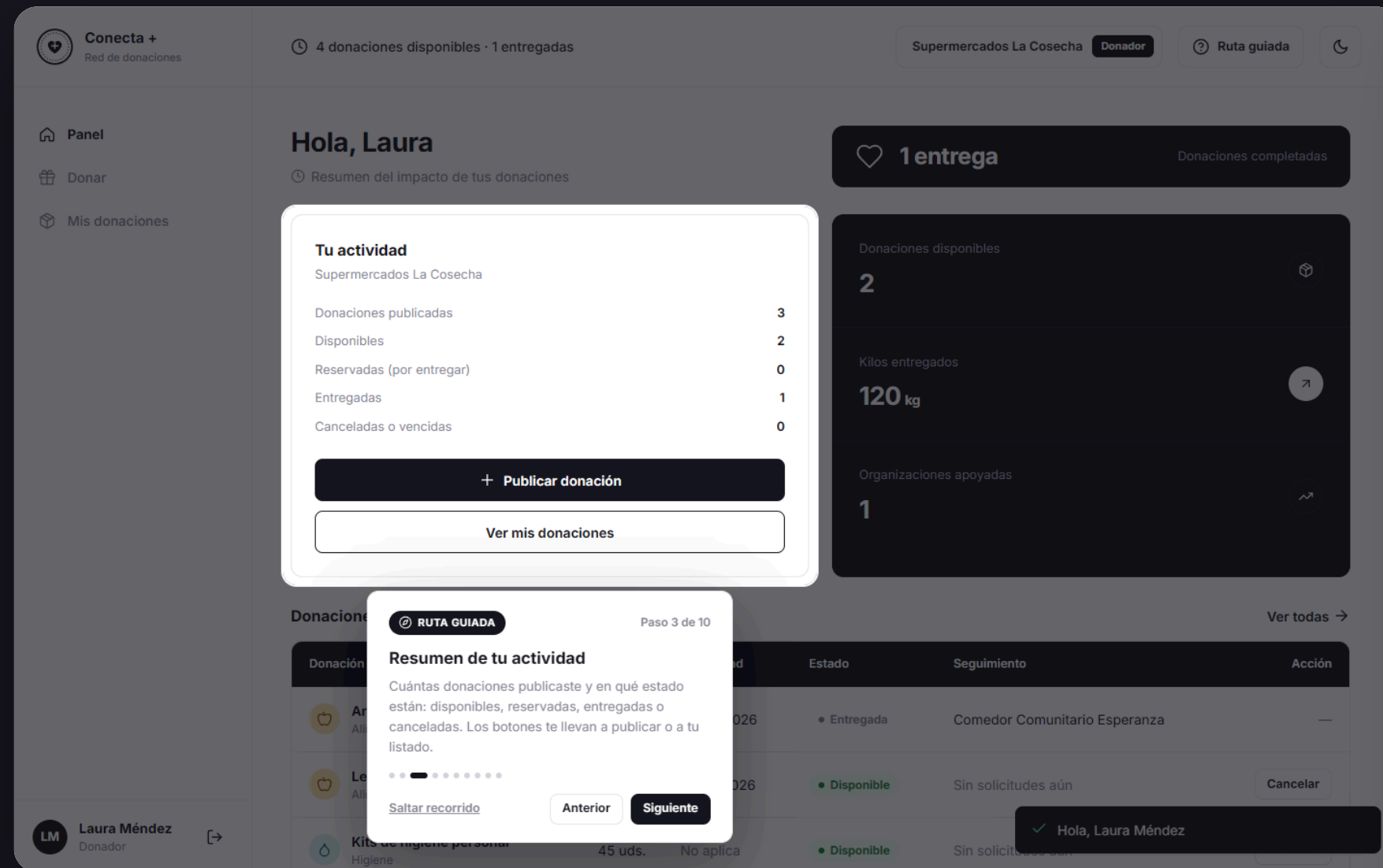
Administrador: aprueba, rechaza o revoca solicitudes.

Ruta guiada en cada panel

Donador · 10 pasos

Beneficiario · 10

Admin · 11



Para qué: que cualquiera aprenda a usar su panel sin manual.

Resalta cada parte y explica qué hace.

Arranca sola la primera vez, también en el celular.

IMPLEMENTACIÓN

Cómo está construida la aplicación



En palabras simples: son tres partes que se hablan entre sí. Todo viaja dentro de una «caja» llamada Docker, que hace que la app funcione igual en cualquier computadora, sin pagar servidores ni bases de datos.

HTML · CSS · JavaScript

Navegador

Para qué: es lo que ves y tocas: pantallas, botones y formularios.



Node.js · Express · JWT

API

Para qué: es el cerebro; revisa quién eres, qué puedes hacer y aplica las reglas.



Memoria + archivo JSON

Almacenamiento

Para qué: es la memoria; guarda usuarios, donaciones y solicitudes.

Saber quién eres



En palabras simples: el sistema necesita saber quién eres cuando inicias sesión y asegurarse de que nadie pueda hacerse pasar por ti.



Pase digital

JWT firmado · expira en 2 h · revocable

Para qué: como una pulsera de evento, te deja entrar sin pedir la contraseña a cada rato. Se anula al cerrar sesión.



Cookie protegida

httpOnly · SameSite=Strict

Para qué: guarda el pase donde ningún código malicioso puede leerlo ni usarlo desde otra página.



Freno a los adivinos

bcrypt · bloqueo tras 5 intentos

Para qué: si alguien intenta adivinar tu contraseña, la cuenta se bloquea 15 minutos. Y las contraseñas se guardan cifradas.

Qué puede hacer cada quien



En palabras simples: cada persona ve y hace solo lo que le corresponde, y nadie puede colar datos o código tramposo.



Formularios estrictos

Validación que rechaza campos extra

Para qué: nadie puede registrarse como administrador mandando datos de más.



Cada quien lo suyo

Permisos por rol y por dueño

Para qué: un donador no ve ni cancela donaciones de otro; el sistema responde como si no existieran.



Sin código escondido

CSP estricta · datos como texto

Para qué: si alguien escribe código en un campo, se muestra como texto y nunca se ejecuta.

Pruebas unitarias con Jest



¿Qué son? Pequeños programas que usan la aplicación solos y avisan si algo se rompe. La **cobertura** dice qué parte del código revisan: 99.8 % es casi todo.

131

pruebas, 0 fallos

99.8%

de líneas

96.9%

de ramas

99.3%

de funciones

Para qué: cada cambio se comprueba en segundos, sin probar todo a mano. La meta mínima era 80 %.

Un robot que revisa y publica cada cambio



En palabras simples: cada vez que se sube un cambio a GitHub, un «robot» lo prueba, lo revisa y lo publica solo. Si algo falla, se detiene y el error no llega a los usuarios.



GitHub Actions · las 5 etapas pasan en verde · github.com/EvanDavidMS/conecta-mas-donaciones

OWASP ZAP: atacamos nuestra propia app



¿Qué es? Una herramienta gratuita que ataca la aplicación como lo haría un hacker, para encontrar los huecos antes que ellos. La usamos con la sesión iniciada, contra todas las funciones.

Inyección SQL

Escribir comandos en un formulario para leer o borrar datos ajenos.

Resistió · 6 variantes

XSS

Esconder código que se ejecuta en la pantalla de otra persona para robar su sesión.

Resistió · 3 tipos

Comandos y rutas

Intentar darle órdenes al servidor o leer sus archivos internos.

Resistió

Resultado del ataque

0 / 0

alertas de riesgo alto / medio

Qué significa: hoy no hay huecos graves que alguien pueda aprovechar.

Hallazgo	Riesgo	Qué significa y qué hicimos
Respuesta inesperada a peticiones raras	Bajo	Algunas rutas respondían de más; ahora devuelven un error claro.
Comentario sospechoso	Info	Una palabra parecía una nota pendiente; se corrigió.
Contenido sin caché	Info	Es a propósito: no deja datos personales guardados.
Almacenamiento del navegador	Info	Solo guarda el tema y si viste la ruta guiada.

SonarQube: un revisor automático del código



¿Qué es? Como un corrector ortográfico, pero para programas: busca errores, huecos y código confuso, y califica de A (mejor) a E.

Qué mide	Antes	Después
Seguridad · qué tan protegido está el código	C	A
Vulnerabilidades · huecos que alguien podría aprovechar	1	0
Code smells · código confuso, difícil de mantener	8	0
Deuda técnica · tiempo para arreglar lo pendiente	40 min	0 min
Bugs · errores que harían fallar la app	0	0



La vulnerabilidad

Una contraseña de ejemplo estaba escrita en el código, a la vista de cualquiera. Ahora está en un archivo de configuración aparte.

Planificado frente a ejecutado

Fase	Plan	Real	Variación
Implementación y seguridad	4 h	7.25 h	+81 %
Pruebas y calidad	3 h	2.75 h	-8 %
Cierre y evaluación	2 h	2 h	0 %
Total	9 h	12 h	+33 %

¿Por qué?

- El alcance pasó de 2 a 3 roles
- La interfaz no estaba estimada
- El cliente pidió la ruta guiada
- El primer pipeline falló en SonarQube

Lo que haríamos igual y lo que cambiaríamos

1 · Seguridad desde el diseño

Validación estricta y frontend sin innerHTML:
ZAP no halló riesgos medios ni altos.

2 · Configuración fuera del código

Una contraseña de demo en el código bajó la
seguridad a C en SonarQube.

3 · Probar el pipeline pronto

Las diferencias entre Windows y Linux (rutas,
permisos) solo aparecieron en CI.

4 · Validar con el cliente temprano

La ruta guiada surgió al final; revisarlo antes
habría permitido planificarla.

PLAN DE MEJORA CONTINUA

Qué haremos, para qué y cómo lo mediremos

Acción	Para qué sirve	Meta medible	Plazo
Pruebas automáticas de las pantallas	Probar la app como lo haría un usuario real	Los 8 flujos principales probados	1 mes
Estándar de calidad más alto	Frenar el código con riesgos antes de publicarlo	Seguridad A; frenar riesgo medio	2 semanas
Avisos por correo	Que nadie espere días por una respuesta	Respuesta en menos de 24 h	1 mes
Alertas de caducidad	Desperdiciar menos comida	Menos del 5 % vencidas al mes	2 meses
Base de datos al crecer	Que la app aguante muchos más usuarios	Más de 1 000 donaciones al mes	Según demanda

IA para predecir futuras donaciones

En palabras simples: aprender del historial qué se dona, dónde y cuándo, para avisar a las organizaciones antes de que algo falte.

Predecir la oferta

Para qué: que las organizaciones planeen con tiempo. Meta: error menor al 20 % a 4 semanas.

Emparejamiento inteligente

Para qué: que cada donación llegue antes a quien más la necesita. Meta: 30 % menos tiempo de asignación.

Gracias.

¿Preguntas?

3 roles con acceso
seguro

99.8 % de cobertura

0 riesgos altos o medios

SonarQube A · A · A